

Project - Low Level Design

on

Travel Crew -

Multi-Agent

Hospitality

System

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Harsh Rai	EN22CS301393
2	Hemant Dhakad	EN22CS301421
3	Gaurav Dwivedi	EN23CS3L1008
4	Goutam Verma	EN22CS301375
5	Harshwardhan Yadav	EN22CS301416

Group Name: Group 12D4

Project Number: AAI-08

Industry Mentor Name:

University Mentor Name:

Academic Year: 2025-2026

Table of Contents

1. Introduction.
 - 1.1. Scope of the document.
 - 1.2. Intended Audience.
 - 1.3. System overview.
2. System Design.
 - 2.1. Application Design.
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. Components Design.
 - 2.5. Key Design Considerations.
 - 2.6. API Catalogue.
3. References.

Introduction

1.1 Scope of the Document

TravelCrew – a Multi-Agent Hospitality System developed as part of the Datagami Agentic AI course (Project AAI-08). It covers the system architecture, application design, process flow, information flow, component-level design, key design considerations, and API catalogue. This document serves as a blueprint for implementation, evaluation, and future enhancement of the system.

1.2 Intended Audience

1. University and Industry Mentors evaluating the project deliverables.
2. Team members responsible for implementing and maintaining the system.
3. Developers who may extend or build upon TravelCrew in future iterations.
4. Academic reviewers assessing the Agentic AI course outputs.

1.3 System Overview

TravelCrew is an AI-powered multi-agent hospitality system. A user submits a natural language travel request (e.g. “5 days in Tokyo for a couple”) and two specialized AI agents collaborate to produce a complete, personalized travel itinerary. The system is built on the CrewAI framework and uses a sequential hand-off protocol:

Researcher Agent: Searches a Chroma DB internal knowledge base and the live web via Serper API to gather hotels, restaurants, attractions, weather, transport, and tips.

Writer Agent: Receives the research output and synthesizes it into a structured day-by-day itinerary with budget estimates and practical tips.

The system is accessible via a Streamlit web interface and is powered by the Groq LLaMA 3.3 70B language model.

System Design

2.1 Application Design

TravelCrew follows a modular, layered architecture. Each file has a single clear responsibility, making the system easy to maintain and extend.

Layer	File	Responsibility
Presentation	<i>app.py</i>	Streamlit UI – accepts user input and displays itinerary output
Orchestration	<i>crew_setup.py</i>	Assembles agents and tasks into a sequential CrewAI workflow
Agent Logic	<i>agents.py</i>	Defines Researcher and Writer agents with roles, goals, and tools
Task Logic	<i>tasks.py</i>	Defines research and writing tasks with context hand-off
Knowledge Base	<i>rag_setup.py</i>	ChromaDB vector store with pre-loaded travel knowledge and retriever

2.2 Process Flow

The end-to-end process flow of TravelCrew from user input to final itinerary:

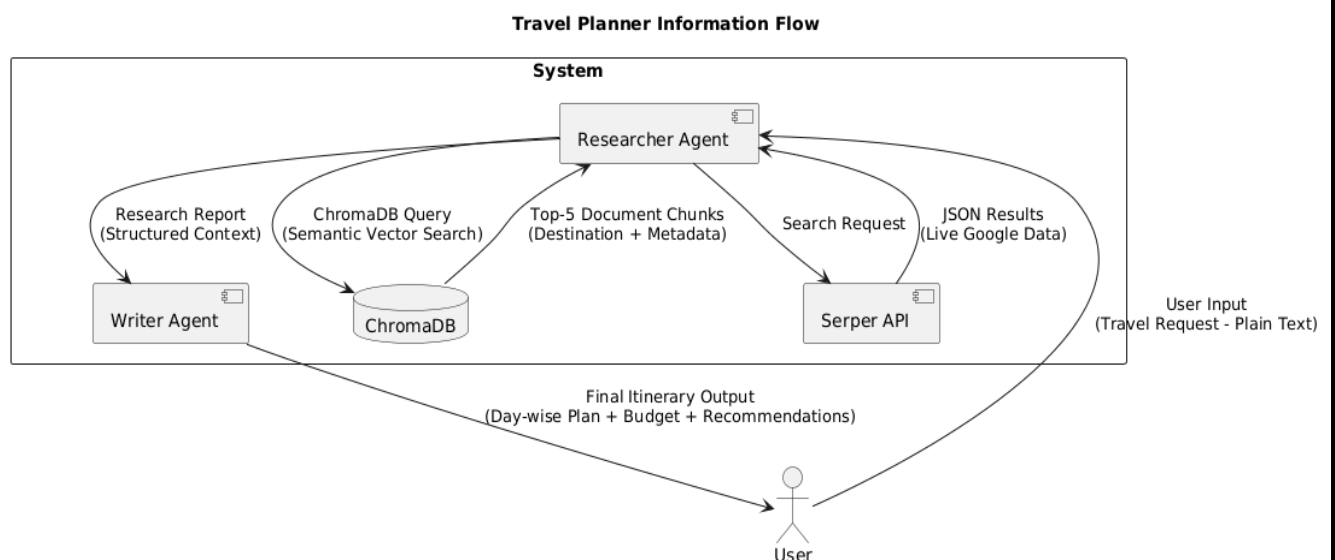
1. User enters a travel request in the Streamlit UI (e.g. “5 days in Tokyo for a couple”).
2. The app calls `hospitality_crew.kickoff(inputs={'travel_request': ...})`.
3. CrewAI starts the sequential process and assigns the Research Task to the Researcher Agent.
4. Researcher Agent invokes `TravelKnowledgeTool` to query ChromaDB for relevant destination data.
5. ChromaDB uses HuggingFace sentence embeddings to perform semantic similarity search and returns top 5 matching documents.
6. Researcher Agent then invokes `SerperDevTool` to supplement with live Google search results.
7. Researcher Agent compiles a comprehensive research report covering hotels, restaurants, attractions, weather, transport, and tips.

8. CrewAI passes the research report to Writer Agent via the context=[research_task] hand-off parameter.
9. Writer Agent synthesizes the research into a structured day-by-day travel itinerary with budget estimates.
10. Final itinerary is returned to the Streamlit UI and displayed to the user.

2.3 Information Flow

Data types and flow between components:

User Input → Researcher Agent → Writer Agent → Final Itinerary Output



User Input: Plain text string representing the travel request.

ChromaDB Query: Semantic vector search query derived from the travel request.

ChromaDB Output: Top-5 matching document chunks with destination and category metadata.

Serper Output: JSON containing live Google search results for current data.

Research Report: Structured text covering all hospitality categories, passed as context to Writer Agent.

Final Output: Formatted multi-section travel itinerary with day-by-day schedule, recommendations, and budget breakdown.

2.4 Components Design

Researcher Agent

The Researcher Agent is the first agent in the sequential pipeline. It is responsible for gathering all destination-specific information required to build the itinerary.

Property	Details
Role	Hospitality Research Specialist
Goal	Gather comprehensive destination data from knowledge base and live web
Tools	TravelKnowledgeTool (ChromaDB RAG), SerperDevTool (Google Search)
LLM	Groq LLaMA 3.3 70B Versatile
Output	Structured research report: hotels, restaurants, attractions, weather, transport, tips

Writer Agent

The Writer Agent receives the Researcher's output via CrewAI's context hand-off and synthesizes it into a professional travel itinerary.

Property	Details
Role	Travel Itinerary Writer
Goal	Synthesize research into a beautifully structured day-by-day itinerary
Input	Full research report via CrewAI context=[research_task] hand-off
Tools	None (relies entirely on research context from Researcher Agent)
LLM	Groq LLaMA 3.3 70B Versatile
Output	Day-by-day itinerary with schedule, hotel picks, dining, budget, and tips

TravelKnowledgeTool (ChromaDB RAG)

A custom CrewAI BaseTool wrapping the ChromaDB retriever. When invoked, it performs semantic similarity search over the pre-loaded travel knowledge base and returns the top 5 most relevant document chunks with metadata labels.

Property	Details
Vector Store	ChromaDB (persist_directory: travel_knowledge_db/)
Embedding Model	sentence-transformers/all-MiniLM-L6-v2 (runs locally via HuggingFace)
Search Type	Semantic similarity search (k=5 results)
Knowledge Coverage	Paris, Tokyo, Bali, India – hotels, restaurants, attractions, weather, transport, tips
Persistence	Auto-persists on first run; loads from disk on all subsequent runs

Key Design Considerations

Agent Specialization

Each agent has a clearly defined role, goal, and backstory ensuring it performs its task optimally without overlap. The Researcher focuses exclusively on data gathering; the Writer focuses exclusively on synthesis and formatting.

Hand-off Protocol

Inter-agent communication is implemented via CrewAI's context parameter. The writing_task is configured with context=[research_task], causing CrewAI to automatically append the full Researcher output to the Writer's input prompt – ensuring zero data loss during the agent transition.

RAG-First Strategy

The Researcher Agent is instructed to always query TravelKnowledgeTool (ChromaDB) before performing web search. This reduces external API calls, improves response speed for known destinations, and ensures grounded answers from the curated knowledge base.

Sequential Process and State Management

CrewAI's Process.sequential mode enforces strict execution order – the Researcher always completes before the Writer begins. CrewAI internally maintains task state so the Writer always receives complete, structured research data.

Security

All API keys (GROQ_API_KEY, SERPER_API_KEY) are stored in a .env file loaded via python-dotenv. The .env file is excluded from version control via .gitignore to prevent credential exposure.

API Catalogue

API / Service	Provider	Usage	Details
Groq Inference API	Groq	LLM for both agents	Model: llama-3.3-70b-versatile, Temp: 0.4
Serper Search API	Serper.dev	Live web search	Google Search results as JSON. Free: 2,500 searches/month
HuggingFace Embeddings	HuggingFace	Text embeddings for RAG	Model: all-MiniLM-L6-v2, runs locally, no API key needed
ChromaDB	Local	Vector store for RAG	Persisted locally in travel_knowledge_db/, no external API required
Streamlit	Local	Web UI server	Runs on localhost:8501, no external API required

References

- [1] CrewAI Inc., "CrewAI Documentation," *CrewAI*, 2024. [Online]. Available: <https://docs.crewai.com>. [Accessed: Mar. 18, 2026].
- [2] Groq Inc., "Groq API Documentation," *Groq*, 2024. [Online]. Available: <https://console.groq.com/docs>. [Accessed: Mar. 18, 2026].
- [3] Chroma Research Inc., "ChromaDB Documentation," *Chroma*, 2024. [Online]. Available: <https://docs.trychroma.com>. [Accessed: Mar. 18, 2026].
- [4] LangChain Inc., "LangChain Documentation," *LangChain*, 2024. [Online]. Available: <https://python.langchain.com>. [Accessed: Mar. 18, 2026].
- [5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP*, 2019. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>. [Accessed: Mar. 18, 2026].
- [6] Serper Technologies, "Serper API Documentation," *Serper.dev*, 2024. [Online]. Available: <https://serper.dev/docs>. [Accessed: Mar. 18, 2026].
- [7] Streamlit Inc., "Streamlit Documentation," *Streamlit*, 2024. [Online]. Available: <https://docs.streamlit.io>. [Accessed: Mar. 18, 2026].